MLOps Foundations: From Notebook to Production

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 12, 2026

Learning Outcomes

By the end of this session, you will be able to:

1. Describe the ML lifecycle from experiment to operated system, and name the failure mode that lives at each stage.
2. Make a training run reproducible: seeds, deterministic operations, environment pinning, and data/config versioning.
3. Track experiments with MLflow or Weights & Biases, and run a systematic hyperparameter search with Optuna.
4. Package a model and serve it behind an API, and choose between batch and online serving.
5. Design basic production monitoring: data drift, concept drift, performance alerts, and retraining triggers.

Table of Contents

Table of Contents (cont.)

Everything So Far Ended at `model.fit()`

Every deck in this course stops at the same place: a trained model, in a notebook, on your laptop.

What we have taught you

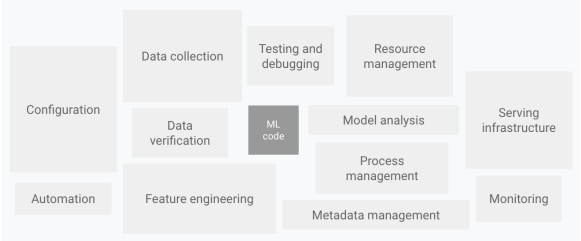
- Split the data, pick a metric
- Fit a model, tune it a bit
- Report a validation score

What nobody taught you

- Which run produced that number?
- Can you rebuild it in six months?
- Who calls the model, and what happens when it is wrong?
- How will you know it stopped working?

MLOps is the set of practices that answers the right-hand column.

The ML Code Is the Small Box



Source: Google Cloud, "MLOps: Continuous delivery and automation pipelines in machine learning," Cloud Architecture Center, Figure 1 — after Sculley et al., *Hidden Technical Debt in Machine Learning Systems*, NeurIPS 2015, Figure 1.

- Sculley et al. (2015): "*Only a small fraction of real-world ML systems is composed of the ML code . . . The required surrounding infrastructure is vast and complex.*"
- The grey box is what this course has been about. Everything around it is what makes it *useful*.

Hidden Technical Debt

The core argument (Sculley et al., NeurIPS 2015)

Building an ML system is fast and cheap. **Maintaining** it is slow and expensive. The debt is *hidden* because it lives at the system level, not in the code, so ordinary code review and unit tests do not find it.

Boundary erosion

- **Entanglement (CACE):** *Changing Anything Changes Everything*. Change one feature's distribution and every other weight shifts.
- **Correction cascades:** a model that patches another model's output — now you cannot improve either one alone.
- **Undeclared consumers:** someone reads your predictions off a log and you never find out.

System-level anti-patterns

- **Glue code:** 95% of the repository is plumbing around a 5% library call.
- **Pipeline jungles:** scrapes and joins accreted over years; nobody can rerun them.
- **Configuration debt:** a hundred flags, no schema, no review.
- **Feedback loops:** the model influences the data it will next be trained on.

What Actually Breaks After Deployment

None of these are modelling problems. All of them are engineering problems.

Symptom	Actual cause
Accuracy fell off a cliff overnight	An upstream team renamed a column; your feature is now all-null
Accuracy decayed slowly over a year	The world changed; your training distribution did not
"Works on my machine"	Unpinned dependency; a minor version bumped a default
Cannot reproduce last quarter's model	The training data was overwritten in place
Two teams report different numbers	Different splits, different seeds, different preprocessing
Nobody noticed for three months	Nothing was monitored except server uptime

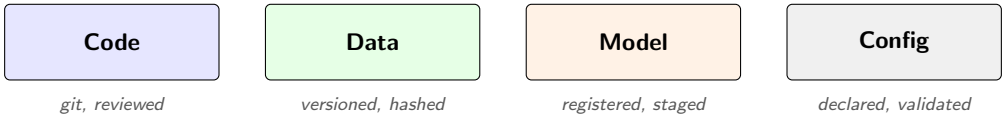
See Paleyes, Urma & Lawrence, "Challenges in Deploying Machine Learning: a Survey of Case Studies," *ACM Computing Surveys*, 2022, arXiv:2011.09926v3, for a catalogue of these from published industry post-mortems.

So What Is MLOps?

Working definition

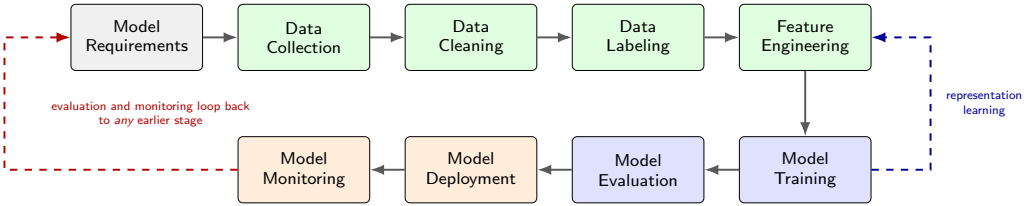
MLOps applies DevOps practices — version control, automated testing, continuous integration and delivery, monitoring — to machine learning systems, where the deliverable is not just code but **code + data + model** together.

A deployment is a *tuple* of all four — not a single commit.



- **Not** a product you buy. **Not** a job title you hire once. It is a set of habits plus a small amount of tooling.
- The habits matter more than the tools. A team with a spreadsheet and discipline beats a team with a platform and none.

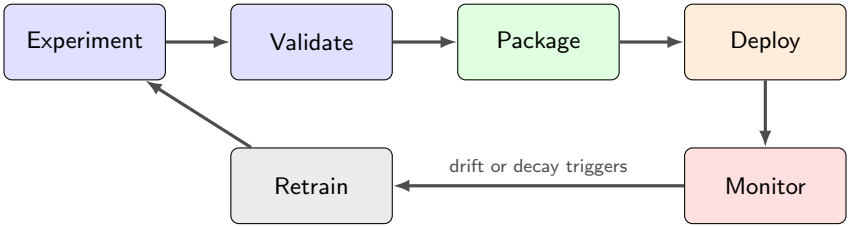
The Lifecycle, Stage by Stage



Stages after Amershi et al., "Software Engineering for Machine Learning: A Case Study," ICSE-SEIP 2019, Figure 1. Paper.

- **Green** is data work, **blue** is model work, **orange** is operations.
- The dashed arrows are why a linear project plan always slips.

The Part This Deck Is About: The Operating Loop



- Each arrow is a place work gets lost. Each box is a place something can silently break.
- **The goal of MLOps is to shorten this loop**, not to add ceremony to it.
- Rule of thumb: if one lap takes a week of manual effort, you will do it twice a year — and your model will spend most of its life stale.

One Failure Mode per Stage

Stage	Characteristic failure	Cheapest defence
Experiment	Results not attributable to any run	Experiment tracking
Validate	Leakage; tuned on the test set	Frozen holdout, nested CV
Package	Environment mismatch; unsafe artifact	Lockfile + container; safe format
Deploy	Training/serving skew in preprocessing	One shared transform, tested
Monitor	Silent degradation, nobody notices	Input and prediction distribution alerts
Retrain	Retrained on drifted, unvalidated data	Data validation before training

Training/serving skew

The single most common production bug in classical ML: the notebook computed a feature one way, the service computes it another way. The model is fine; the inputs are not.

Who Does What

- **Data engineer** — pipelines, warehouses, schemas. Owns whether the data *arrives* and is well-formed.
- **Data scientist** — framing, features, models, evaluation. Owns whether the model is *right*.
- **ML engineer** — packaging, serving, retraining. Owns whether the model *runs*.
- **Platform / SRE** — infrastructure, capacity, on-call. Owns whether the service *stays up*.
- **Product owner** — the decision the prediction supports. Owns whether the model is *worth* running.

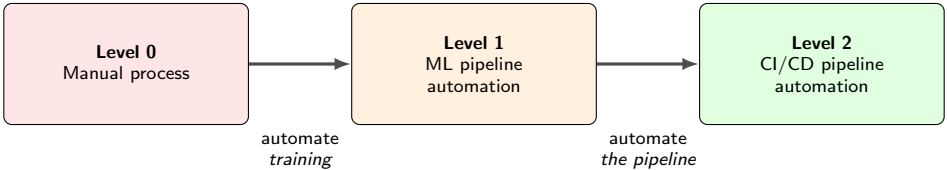
In a small team one person wears all five hats. That is fine — but the *handoffs* still exist, and they still need to be written down.

The handoff that kills projects

"The data scientist throws a notebook over the wall to the engineer." The engineer rewrites the preprocessing, the numbers change, and nobody can say which version is correct.

MLOps Maturity: Three Levels

Google's framing. Useful mostly because it names what you are allowed to not have yet.



- **Level 0** deploys a *model*. **Level 1** deploys a *training pipeline*. **Level 2** deploys *changes to that pipeline*, automatically and tested.
- Most organisations that believe they are at level 2 are at level 0 with a scheduled job.
- **Maturity is a cost, not a virtue.** Buy the level your problem justifies: how often must the answer change, what does a wrong prediction cost, and how many people must be able to reproduce the result?

Source: Google Cloud, "MLOps: Continuous delivery and automation pipelines in machine learning," Cloud Architecture Center, 2020.

Level 0: Manual Process

What it looks like

- Every step is manual, script-driven and interactive — data prep, training and validation all live in a notebook.
- A hard split between the people who *build* the model and the people who *serve* it: only the artifact is handed over.
- Infrequent release iterations — a handful of model changes a year.
- No CI/CD: a model change is not a code change that anyone tests.
- No active performance monitoring in production.

When level 0 is the right answer

- The model is retrained rarely, or never.
- The world underneath it changes slowly.
- One person owns it end to end.

Most pilots and proofs of concept should stay here.

The trap

Level 0 fails the moment the world moves faster than your release cadence — and you have no instrument that tells you it has.

Level 1: Pipeline Automation → Level 2: CI/CD

Level 1: ML pipeline automation

Goal: *continuous training* — the model retrains in production on fresh data.

- Orchestrated, modular, containerised pipeline steps.
- **Data validation** and **model validation** steps inside the pipeline.
- **Experimental-operational symmetry**: the pipeline you develop with is the pipeline that runs in production.
- A metadata store; optionally a feature store.
- Triggers: on a schedule, on new data, on demand, or on measured degradation.

Still manual: deploying a *new* pipeline.

Level 2: CI/CD pipeline automation

Goal: rapidly and safely explore new features, architectures and hyper-parameters.

- Source control triggers an automated **build and test** of pipeline components.
- **Continuous delivery** pushes the new pipeline to production.
- The deployed pipeline then performs continuous training on its own.
- Model registry and end-to-end lineage as first-class infrastructure.

The unit of delivery is the pipeline, not the model.

- **Quarterly retrain, one owner, low stakes** → level 0 plus experiment tracking is enough.
- **Weekly retrain, several owners** → level 1, or you will spend your life running notebooks by hand.
- **Daily retrain, high-stakes or revenue-critical** → level 2, or you will not be able to prove what shipped.

Level definitions from Google Cloud, "MLOps: Continuous delivery and automation pipelines in machine learning," Cloud Architecture Center, 2020.

“It Worked Last Tuesday”

A result you cannot rebuild is a rumour, not a result.

1. Repeatable

You, same machine, same code, same data, next week → same number.
This is the bar for this course.

2. Reproducible

A colleague, different machine, your code and data → the same number.
This is the bar for a team.

3. Replicable

A stranger, their own implementation → the same *conclusion*.
This is the bar for science.

- Level 1 is almost free and almost nobody does it. Get level 1 before worrying about the rest.
- Reproducibility is not a research nicety — it is what lets you **roll back** when a deployment goes wrong.

Five Places Nondeterminism Hides

1. **Random number generators.** Weight initialisation, shuffling, dropout, data augmentation, train/test splits, bootstrap sampling in random forests.

2. **Parallelism.** Floating-point addition is not associative; sum the same numbers in a different order on a different thread and you get a different last bit — which compounds.

3. **Library and hardware versions.** A minor release changes a default; a different GPU picks a different convolution kernel.

4. **The data itself.** A table that is updated in place has no version. "The training set" is not a thing unless you make it one.

5. **Configuration.** The flag you changed by hand at 11pm and did not commit.

Order of attack

Seeds and data versioning buy you the most for the least effort. Bit-exact determinism on GPU is the most expensive and usually the least valuable.

Seeding: Every Library, Every Time

```
# PYTHONHASHSEED must be set BEFORE the interpreter starts, so it goes in the
# launcher, not in the script:   export PYTHONHASHSEED=42
import random, numpy as np, torch

def set_seed(seed: int = 42):
    random.seed(seed)           # Python stdlib
    np.random.seed(seed)        # NumPy legacy global RNG
    torch.manual_seed(seed)     # CPU *and* all CUDA devices

set_seed(42)

# scikit-learn: pass random_state explicitly -- never rely on the global seed
train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
RandomForestClassifier(n_estimators=500, random_state=42)
```

- **Never rely on the global NumPy seed for scikit-learn.** Anything left at `random_state=None` falls back to NumPy's *global* RNG, so its result depends on whatever consumed random numbers first. Pass `random_state` explicitly, everywhere.
- Any subprocess gets a fresh interpreter and therefore a fresh RNG state — PyTorch `DataLoader` workers, `joblib` parallel jobs, Spark executors. Each needs its own seeding hook.
- **Log the seed as a tracked parameter.** A seed you cannot look up later is not a seed.

The Limits of Determinism

What frameworks actually promise

PyTorch: "Completely reproducible results are not guaranteed across PyTorch releases, individual commits, or different platforms" — nor, it adds, between CPU and GPU executions, even with identical seeds.

And what it costs

"Deterministic operations are often slower than nondeterministic operations, so single-run performance may decrease for your model."

The pragmatic policy

- Turn strict determinism **on** while chasing a regression — it turns a statistical question into a deterministic one.
- Turn it **off** for long production training runs and instead report the **mean and spread over 3–5 seeds**.
- **A result that only holds for one seed is not a result.** Before believing a $+0.5\%$ improvement, measure how much the score moves when you change nothing but the seed.

Source: PyTorch documentation, "Reproducibility," docs.pytorch.org/docs/stable/notes/randomness.html. The deep-learning switches themselves (`use_deterministic_algorithms`, cuDNN flags, `DataLoader` worker seeding, checkpoint discipline) are covered hands-on in the computer-vision course; here we stay framework-agnostic.

Pinning the Environment

The ladder, weakest to strongest

1. `pip install pandas` — no pin. Broken by definition.
2. `requirements.txt` with `==` — pins your *direct* dependencies only.
3. **A lockfile** (`uv.lock`, `poetry.lock`, `conda-lock.yml`) — pins the whole transitive graph, with hashes.
4. **A container image** referenced by digest — pins the OS, the CUDA driver stack and the Python packages together.

Steps 3 and 4 are the ones that matter. Steps 1–2 fail six months later, quietly.

```
# resolve once, commit the lockfile
uv lock
uv sync --frozen

# record what actually ran
pip freeze > run_env.txt
nvidia-smi --query-gpu=name,driver_version
      \
           --format=csv > run_gpu.txt
```

Minimum viable habit: log the lockfile hash and the git commit as parameters of every training run. Two lines of code; saves entire quarters.

Versioning the Data

The problem

Git is built for text files of a few kilobytes. Your training set is 40 GB of Parquet, and someone just overwrote it in place. The code from March still runs; it just trains on different data now.

- **The idea (DVC, git-lfs, lakeFS, Delta Lake all share it):** commit a small *pointer* that contains a content hash; keep the bytes somewhere cheap. Checking out an old commit now checks out the matching data.
- **The cheap version, if you adopt no tool at all:** write data to immutable, dated paths (`s3://bucket/train/2026-03-14/`) and never overwrite. Log the path with the run.
- **Never** version a mutable database table by referring to "the customers table". Refer to a snapshot, or to a query plus an as-of timestamp.

Configuration as code

- Hyperparameters and paths belong in a *declared* config file (YAML, or a framework such as Hydra), not in argparse defaults scattered across three scripts.
- The config is committed, diffable and reviewable. "What changed between run 41 and run 42?" becomes a `diff`.
- Validate the config on load — a typo in a key name should crash, not silently fall back to a default.

Log with every run

- git commit hash **and** whether the tree was dirty
- the full resolved config
- random seed(s)
- dataset identifier or content hash
- lockfile hash / image digest
- metrics, and the evaluation code version

The community version of this list is the *ML Reproducibility Checklist*: Pineau et al., "Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program)," 2020, arXiv:2003.12206v4.

Good news: once you adopt experiment tracking — the next section — five of these six items are logged for you automatically.

The Spreadsheet Everyone Starts With

`results_final_v3_REAL.xlsx`

run	lr	acc
exp1	0.01	0.83
exp1b	0.01	0.86
exp2_new	0.001	0.91
final	?	0.94
final2	?	0.93

Which commit? Which data? Which split? Which preprocessing?

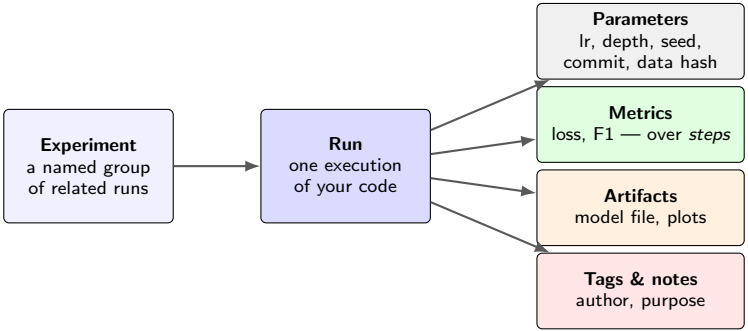
What always goes wrong

- The best number belongs to a run nobody can rebuild.
- Two rows differ by something that was never written down.
- Someone edits a cell after the fact.
- The file lives on one laptop.

The fix

Make the *run* a first-class object that the code writes to, automatically, and that nobody can retro-edit.

The Data Model: What a "Run" Contains



- **A run is append-only by convention.** You add to it, you do not edit it — so the record cannot drift away from what actually happened.
- Log metrics **per step**, not only at the end: a learning curve diagnoses ten times more than a single number.
- Log the **inputs** (commit, config, data hash) as parameters. Without them a run is a number without provenance.

MLflow: Four Components

Component	What it is
Tracking	An API and UI for logging parameters, code versions, metrics and artifacts when you run ML code, and for visualising the results afterwards.
Projects	A standard format for packaging and sharing reproducible data-science code, with its entry points and environment declared, so someone else can rerun it.
Models	A standard packaging format with "flavours". Anything supporting the `python_function` flavour can be served as a REST endpoint, batch-scored in Spark, or exported to a cloud serving platform.
Model Registry	A centralised model store with versioning, aliases, tags and annotations, plus lineage back to the run that produced each version.

- They are designed to work together but each one is usable alone. **Start with Tracking** — it is three lines of code and it is where all the value is at first.
- MLflow is open source and runs entirely on your laptop: no account, no network, no cost.

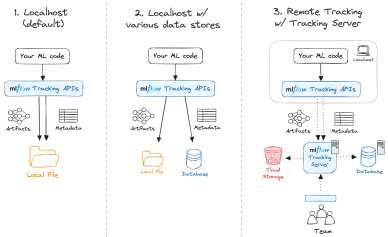
Source: MLflow documentation, mlflow.org/docs/latest/ml. See also Zaharia et al., "Accelerating the Machine Learning Lifecycle with MLflow," *IEEE Data Engineering Bulletin* 41(4), 2018.

Tracking in Practice: The Whole API You Need

```
import mlflow, mlflow.sklearn

mlflow.set_experiment("churn-baseline")

with mlflow.start_run(run_name="rf-depth-sweep"):
    mlflow.log_params({"n_estimators": 500, "max_depth": 12, "seed": 42})
    mlflow.log_param("data_version", "2026-03-14")

    model = RandomForestClassifier(n_estimators=500, max_depth=12,
                                   random_state=42).fit(X_tr, y_tr)

    proba = model.predict_proba(X_va)[:, 1]
    mlflow.log_metric("val_f1",  f1_score(y_va, model.predict(X_va)))
    mlflow.log_metric("val_auc", roc_auc_score(y_va, proba))
    mlflow.log_artifact("confusion_matrix.png")
    mlflow.sklearn.log_model(model, name="model")
```

- `mlflow ui` then serves everything on `localhost:5000`. Nothing else to install.
- **Autologging** (`mlflow.autolog()`) captures parameters, metrics and the model for many frameworks with one line — useful, but log the *data version* and *seed* yourself; no autologger can guess those.

Where Does the Tracking Data Actually Live?



Source: MLflow documentation, "MLflow Tracking," mlflow.org/docs/latest/ml/tracking.

- **Metadata** (params, metrics, tags) goes to a database; **artifacts** (model files, plots) go to object storage. They are separate stores for a reason: one is small and queried, the other is large and streamed.
- Solo work: local files, zero setup. Team work: one shared tracking server, so "my numbers" and "your numbers" live in the same place.

What You Get Back

Source: MLflow documentation, "MLflow Tracking" — metric view of a single run, mlflow.org/docs/latest/ml/tracking.

- Sort and filter runs by any logged metric or parameter; overlay learning curves across runs.
- The payoff arrives about three weeks in, when someone asks "why is production worse than your slide?" and you can answer in thirty seconds.

MLflow and Weights & Biases, Side by Side

	MLflow	Weights & Biases
Model	Open source (Apache 2.0), self-hosted or managed	Hosted SaaS; self-hosting available
Cost	Free; you run the server	Free tier for personal use (corporate use excluded); free Pro licences for academic research
Setup	`pip install mlflow` — works offline	Account plus API key; logs to the cloud by default
Strength	Registry, packaging, deployment story; lives inside your infrastructure	Collaboration, reports, rich interactive dashboards, built-in sweeps
HPO	Bring your own (Optuna, Ray Tune)	`wandb sweeps` built in

- Both cover **runs, params, metrics, artifacts, registry**. The instrumentation in your training script is a handful of lines either way — switching later is cheap.
- **Choose on constraints, not features:** can your data leave the building? Do you have someone to run a server? Is the audience your team or your customer?
- Others in the same shape: Neptune, Comet, ClearML, Aim, and the tracker built into most cloud ML platforms.

Sources: MLflow documentation; W&B pricing (accessed 2026).

Comparing Runs Honestly

Dishonest but common

- Report the best of 50 runs on the validation set as if it were an unbiased estimate.
- Compare your tuned model against an untuned baseline.
- Change three things at once and attribute the gain to your favourite one.
- Report one seed.
- Quietly drop the runs that did not work.

What to do instead

- Keep a **frozen test set** touched once, at the end.
- Give the baseline the *same* tuning budget.
- Change one thing per run; the tracker makes this cheap.
- Report mean $\pm$ spread over 3–5 seeds.
- Keep failed runs, tagged. They are evidence.

The baseline rule

Before any model goes to production, log a run for the **dumbest possible baseline**: predict the majority class, predict last week's value, use the existing business rule. Surprisingly often it wins — and that is a result worth knowing before you build a serving stack.

Where We Left Off

From hyperparameters to search

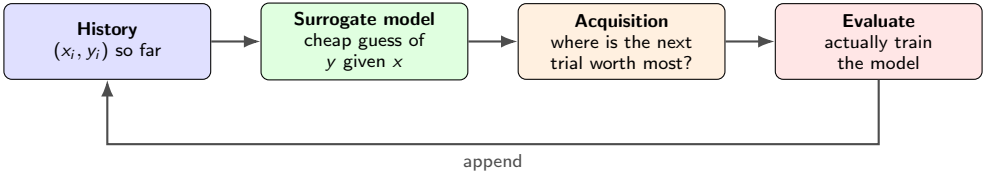
The course has already introduced *what* a hyperparameter is (structural versus optimisation). The two baseline strategies for choosing them: **grid search** enumerates a discretised product of ranges, **random search** draws configurations independently from them, and both score each candidate on a validation split.

	Grid search	**Random search**
Cost	Exponential in the number of parameters	You choose the budget
Uses past trials	No	No
Continuous ranges	Awkward — you discretise	Natural
Good when	2–3 parameters, few values each	A first sweep of anything

The gap

Both are **memoryless**: trial 90 knows nothing about trials 1–89, and a trial that is obviously hopeless at epoch 2 still runs to epoch 50. **Bayesian search** fixes the first, **pruning** the second.

Sequential Model-Based Optimisation: The Idea



- Training one configuration is *expensive*; evaluating a surrogate is *free*. So spend compute on the surrogate to decide where to spend compute on training.
- The acquisition function balances **exploitation** (near the best point seen) against **exploration** (where the surrogate is uncertain).
- Gaussian-process Bayesian optimisation is the textbook version. **TPE** is the version that scales to messy, conditional, mixed-type search spaces — which is what real projects have.

Tree-structured Parzen Estimator (TPE)

The trick

Instead of modelling $p(y \mid x)$ — "how good is this configuration?" — TPE models $p(x \mid y)$: "what do *good* configurations look like?"

1. Split the finished trials into a **good** group (the best objective values) and the **rest**.
2. For each parameter, fit a density $\ell(x)$ to the good group and $g(x)$ to the rest — in Optuna these are Gaussian mixture models.
3. Sample candidates and keep the x that maximises the ratio

$$\frac{\ell(x)}{g(x)}.$$

4. "Looks like the winners, unlike the losers."

Practical notes

- Optuna's default sampler for single-objective studies.
- Its first `n_startup_trials` (default **10**) are *random* — it has nothing to model yet.
- Documented sweet spot: roughly **100–1000 trials**.
- Handles float, int and categorical parameters, and conditional spaces.

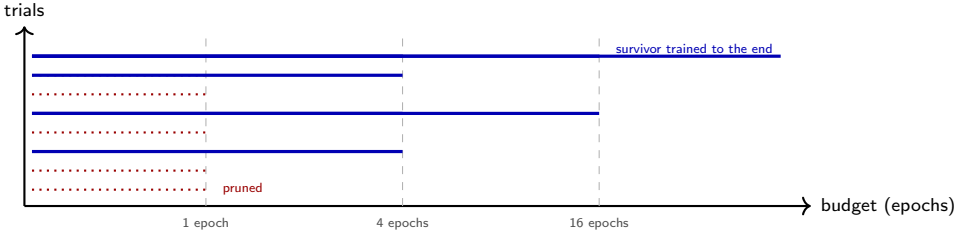
Sources: Bergstra et al., "Algorithms for Hyper-Parameter Optimization," NIPS 2011; Optuna documentation, `TPESampler`.

Optuna: Define-by-Run

```python
import optuna

def objective(trial):
    # the search space is *built by running Python*, so it can branch
    model_name = trial.suggest_categorical("model", ["rf", "gbm"])
    if model_name == "rf":
        clf = RandomForestClassifier(
            n_estimators = trial.suggest_int("n_estimators", 100, 1000, step=100),
            max_depth    = trial.suggest_int("max_depth", 2, 32, log=True),
            random_state = 42)
    else:
        clf = HistGradientBoostingClassifier(
            learning_rate = trial.suggest_float("lr", 1e-3, 3e-1, log=True),
            max_leaf_nodes= trial.suggest_int("leaves", 8, 256, log=True),
            random_state  = 42)
    return cross_val_score(clf, X, y, cv=5, scoring="f1_macro").mean()

study = optuna.create_study(direction="maximize")   # default sampler: TPE
study.optimize(objective, n_trials=100)
print(study.best_params, study.best_value)
```

- **Define-by-run** is Optuna's headline design principle: the search space is constructed dynamically as the objective executes, so an `if` statement gives you a conditional space for free.
- `log=True` for anything spanning orders of magnitude (learning rate, regularisation strength). Sampling learning rate uniformly in $[10^{-5}, 10^{-1}]$ wastes 99% of your budget above 10^{-2}.

Pruning: Stop Wasting Compute on Hopeless Trials



Successive halving / ASHA

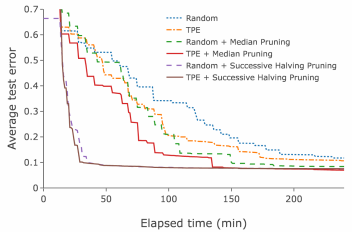
- Give every trial a small budget. At each "rung", promote only the top $1/\eta$ and kill the rest.
- Optuna's `SuccessiveHalvingPruner` implements the *asynchronous* version (ASHA): a trial is promoted as soon as it qualifies, so workers never idle.
- Default reduction factor $\eta = 4$.

Which pruner

- `MedianPruner` — Optuna's default. Prune if you are below the median of finished trials at this step. Gentle, safe.
- `SuccessiveHalvingPruner` — aggressive, best under a fixed wall-clock budget.
- `HyperbandPruner` — runs successive halving at several starting budgets to hedge the "too aggressive" risk.

Sources: Optuna documentation, `SuccessiveHalvingPruner`; Li et al., "A System for Massively Parallel Hyperparameter Tuning," MLSys 2020, arXiv:1810.05934v5; Li et al., "Hyperband," *JMLR* 18, 2018, arXiv:1603.06560v4.

Does It Actually Help? Yes, and Mostly Because of Pruning



Source: Akiba, Sano, Yanase, Ohta & Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," KDD 2019, arXiv:1907.10902v1, Figure 11(a). Simplified AlexNet on SVHN; average test error against *wall-clock* time.

- Read the x-axis carefully: this is error against **elapsed time**, not against number of trials. That is the axis that costs money.
- TPE beats random search at equal time — but the two curves that collapse almost immediately are the **pruned** ones. Pruning is the bigger win.
- In the same paper's RocksDB case study, a 4-hour budget explored **937** configurations with pruning versus **39** without.

Search-Space Design, and When HPO Is Wasted Compute

Designing the space

- **Few parameters, wide ranges** beats many parameters with narrow ones.
- Log scale for anything multiplicative.
- Bound the space with physics and budget, not superstition: if a depth of 40 will not fit in memory, do not offer it.
- If the best value sits on a **boundary**, your range was wrong — widen it and rerun.
- Tune the *pipeline*, not only the model: imputation strategy and scaler often matter more than `n_estimators`.

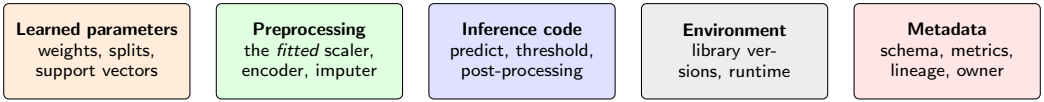
When to skip HPO entirely

- **The baseline is not built yet.** Tune nothing until a dumb model is logged.
- **The validation set is small.** With 200 samples, 500 trials just find the configuration that best overfits your split.
- **The gain is not decision-relevant.** $+0.3$ F1 points that change no business action is compute set on fire.
- **The data is the bottleneck.** A week of labelling usually beats a week of tuning.

Guard the test set

The HPO objective must be computed on **validation** data — ideally cross-validated. A search of 500 trials against a single small split is a very efficient overfitting machine.

“Ship the Model” — Ship *What*, Exactly?



- Forgetting the **fitted** preprocessing is the classic production bug: a scaler refitted on the serving batch silently rescales everything and the model quietly degrades.
- Forgetting the **input schema** means you discover a renamed column at 3am rather than at deploy time.
- Forgetting the **owner** means nobody is paged when it breaks.

Rule

Serialise the **whole pipeline** — `Pipeline([("scale", ...), ("clf", ...)])` — never the estimator alone. One artifact, one `predict`, no room for skew.

Serialisation Formats: What to Save It As

Format	Stores	Good	Bad
`pickle` / `joblib`	Arbitrary Python objects	Works for anything; zero friction	**Executes code on load**; breaks across library versions
`skops`	scikit-learn estimators	Contents inspectable before loading; safer than pickle	scikit-learn ecosystem only
`safetensors`	Tensors + a JSON header	No code execution; fast zero-copy load	Tensors only — no pipeline logic
`ONNX`	A computation graph	Serve without Python; portable across runtimes	Not every estimator or op converts
Framework native	`.pt`, `SavedModel`, `.ubj`	Full fidelity; official support	Ties you to that framework's version

The pickle warning, in scikit-learn's own words

"`pickle` (and `joblib` and `cloudpickle` by extension) has many documented security vulnerabilities by design and should only be used if the artifact ... is coming from a trusted and verified source. You should never load a pickle file from an untrusted source, similarly to how you should never execute code from an untrusted source."

Source: scikit-learn documentation, "Model persistence," scikit-learn.org/stable/model_persistence.html.

Containers, in One Slide

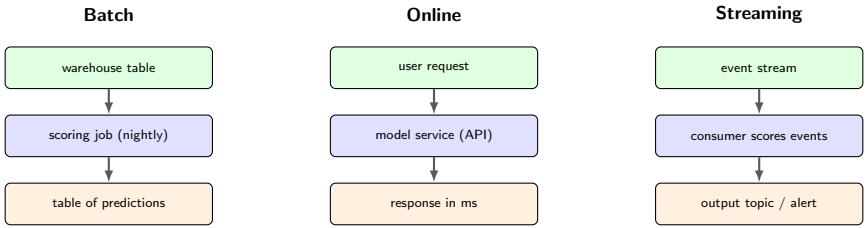
```
FROM python:3.12-slim

WORKDIR /app
COPY requirements.lock .
RUN pip install --no-cache-dir \
        --require-hashes -r requirements.
            lock

COPY serve.py model.skops ./
EXPOSE 8000
CMD ["uvicorn", "serve:app", \
     "--host", "0.0.0.0", "--port", "8000"]
```

- A container freezes the OS libraries, the Python version and the packages **together**. That is the part a lockfile alone cannot do.
- Refer to images by **digest** (`@sha256:...`), not by the tag `latest`. Tags move; digests do not.
- **Do not bake giant model files into the image** if they change often — pull them from the registry at start-up, by version.
- GPU serving adds a CUDA-matched base image; the driver on the host must be new enough.

What a container does *not* give you

Reproducible *training*. Same image, unseeded run, different model. Containers pin the environment; seeds and data versions pin the result.

Batch, Online, Streaming



	Batch	Online	Streaming
Latency	Hours	Milliseconds	Seconds
Cost	Lowest — one big job	Highest — always-on capacity	In between
Failure	Rerun the job	Users see errors	Backlog grows
Use when	The input exists before the question	The input arrives with the question	Continuous events, bounded delay

Default to batch. If a nightly table of predictions solves the problem, build that: an order of magnitude cheaper to run and to operate, and it fails without waking anyone.

A Minimal Online Service

```python
from fastapi import FastAPI
from pydantic import BaseModel
import pandas as pd, skops.io as sio
app = FastAPI()
UNKNOWN = sio.get_untrusted_types(file="model.skops") # inspect, then trust
MODEL = sio.load("model.skops", trusted=UNKNOWN)      # the fitted Pipeline
MODEL_VERSION = "churn-clf/7"       # read from the registry at start-up
class Request(BaseModel):           # the schema IS the contract
    tenure_months: int
    monthly_charges: float
    contract_type: str
@app.get("/healthz")                # liveness: is the process up?
def healthz(): return {"status": "ok"}
@app.post("/predict")
def predict(req: Request):
    X = pd.DataFrame([req.model_dump()])   # same columns as at training time
    p = MODEL.predict_proba(X)[0, 1]
    return {"churn_probability": float(p), "model_version": MODEL_VERSION}
```

- **Return the model version with every prediction.** Without it you cannot attribute a bad outcome to a model later.
- **Load the model once at start-up**, never per request. **Validate the input schema** — a typed request body rejects the renamed column at the door.
- `/healthz` tells the orchestrator the process is alive, not that the model is any good — that is monitoring's job.

Protocols and Latency Budgets

REST/JSON vs gRPC

- **REST + JSON:** human-readable, debuggable with `curl`, works with every client. Verbose; parsing costs real time on large payloads.
- **gRPC + protobuf:** compact binary, schema-checked, streaming, faster for high-volume internal traffic. Harder to inspect.
- **Pick REST first.** Move to gRPC when profiling says serialisation is your bottleneck — rarely true for a scikit-learn model.

Budget the whole request

Network in/out	5–20 ms
Feature lookup (DB / feature store)	5–50 ms
Preprocessing	1–5 ms
Model forward pass	**1–10 ms**
Post-processing, logging	1–5 ms

The model is often the *smallest* term. Optimise what you measured, not what you assumed.

Quote p95 and p99, never the mean. The tail is what users experience and what times out upstream.

Throughput vs latency

Batching requests on the server raises throughput and *raises* latency. Pick which one the product actually needs, then set the batch window accordingly.

CPU or GPU? The Serving Economics

	CPU serving	GPU serving
Fits	Trees, linear models, small nets, most tabular ML	Large nets, transformers, image and audio models
Cost profile	Cheap, scales horizontally, scale-to-zero is easy	Expensive per hour; idle time is pure waste
Wins when	Requests are small and sporadic	Requests are large or can be *batched*
Watch out for	Thread oversubscription inside the container	Low utilisation — a GPU at 5% is a very costly CPU

- **For the classical ML in this course, CPU is almost always right.** A gradient-boosted tree does not benefit from a GPU at inference.
- Before buying a GPU, try: fewer features, a smaller model, ONNX Runtime, or simply caching repeated inputs. These are usually larger wins than new hardware.
- *Going deeper:* model compression for efficient inference (quantisation, pruning, distillation) and multi-GPU/multi-node serving are both covered in the NLP course. We stay framework-level here.

Four Ways a Deployed Model Goes Wrong

What	Looks like	Timescale
Upstream breakage	A feature becomes all-null, all-zero, or wrongly typed	Instant, catastrophic
Data drift	$P(X)$ moves: new customers, new devices, new regions	Weeks to months
Concept drift	$P(Y \mid X)$ moves: the same inputs now imply a different answer	Days to years
Model staleness	Nothing drifted much, but the model has not seen a year of new data	Slow and boring

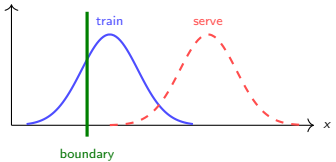
- **Upstream breakage is the most common and the most fixable.** It is a data engineering bug, and a schema check catches it in minutes rather than months.
- The other three cannot be prevented — only detected and answered with a retrain.
- **Server uptime tells you none of this.** A model service returning 200s at 8 ms with catastrophically wrong predictions is a perfectly healthy service.

Drift, Formally

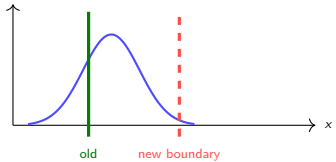
The model learned a joint distribution

Training gave you $P_{\text{train}}(X, Y) = P(X)\,P(Y \mid X)$. Deployment assumes $P_{\text{serve}} = P_{\text{train}}$. Drift is any way that assumption fails.

Covariate shift: $P(X)$ **moves**



Concept drift: $P(Y \mid X)$ **moves**



- **Left:** the inputs moved but the rule is unchanged. Your model may still be right — it is just being asked questions it was not trained on.
- **Right:** the inputs look identical and the *answer* changed. Nothing in the input distribution will tell you. Only labels will.
- **This is why input monitoring alone is not enough** — and why it is still worth having, because it is the only signal available immediately.

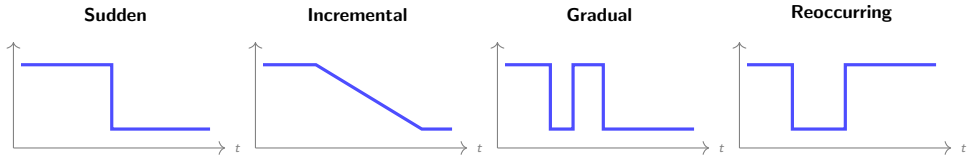
Which Kind Is It? Worked Examples

Situation	Diagnosis
A demand forecaster trained pre-pandemic, run during a lockdown	Both — $P(X)$ and $P(Y \mid X)$ moved together
A fraud model: fraudsters read your rules and adapt	Concept drift, *adversarial* and fast
A churn model after marketing enters a new country	Covariate shift — new customer mix, same mechanism
A sensor is recalibrated and now reports Celsius, not Fahrenheit	Upstream breakage disguised as drift
A recommender trained on its own logged clicks	Feedback loop — the model is shifting its own $P(X)$

Diagnose before you retrain

Retraining on a broken pipeline bakes the breakage into the model. **Always check for upstream breakage first** — it is the cheapest hypothesis and the most common cause.

Drift Comes in Shapes



- **Sudden** — a policy change, a competitor's launch, a pipeline rewrite. Easy to detect, hard to prevent.
- **Incremental / gradual** — the dangerous ones. Any single week looks fine; the year does not. Detected only by comparing against a *fixed* reference window.
- **Reoccurring** — seasonality. **Do not retrain on this.** Give the model the calendar as a feature instead, or you will chase your own tail every December.
- A single anomalous window is an **outlier**, not drift. Do not retrain on one bad Tuesday.

Taxonomy after Gama, Zliobaite, Bifet, Pechenizkiy & Bouchachia, "A Survey on Concept Drift Adaptation," *ACM Computing Surveys* 46(4), Article 44, 2014, DOI:10.1145/2523813, Figure 2.

Detecting Drift Before the Labels Arrive

What you can measure today

- **Schema and range checks** — type, nullability, allowed categories, min/max. Cheap, and catches the common case.
- **Per-feature distribution distance** against a fixed reference window: Kolmogorov–Smirnov for continuous, chi-square or PSI for categorical.
- **Prediction distribution.** If your model predicted "churn" for 4% of users all year and today it says 19%, something changed — even if you cannot yet say whether it was right.
- **Confidence / entropy** of predictions, and the rate of out-of-range inputs.

Population Stability Index

$$\mathrm{PSI} = \sum_b (p_b - q_b) \ln \frac{p_b}{q_b}$$

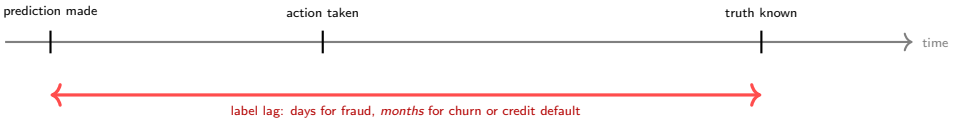
over bins b of a feature; p is the reference, q the current window.

Credit-scoring rule of thumb: < 0.1 stable, 0.1–0.25 moderate shift, > 0.25 significant. A convention, not a theorem — calibrate it on your own history.

The large-n trap

With a million rows per day, every statistical test rejects. Alert on **effect size** and on a **trend**, not on a p-value.

The Label-Lag Problem



- You cannot compute accuracy in production until the labels arrive. For a 12-month loan default model, that is a **12-month** feedback delay.
- **Proxies to use in the meantime:**
  - input and prediction distribution monitoring (previous slide);
  - **business metrics** that respond faster — click-through, override rate, complaint volume, manual-review queue length;
  - **human spot-checks**: label a small random sample every week, deliberately. A hundred labels a week is a real accuracy estimate.
- **Beware biased feedback:** if you only ever see the outcome for applicants you approved, your production "accuracy" is measured on a censored sample.

Alerts People Act On, and Retraining Triggers

Alerting rules that survive contact

- Every alert names an **owner** and a **first action**. An alert with neither will be muted within a month.
- Page on **breakage**; e-mail a weekly digest for **drift**. They are different urgencies.
- Alert on a **sustained** threshold crossing (e.g. 3 days), not on one noisy window.
- Track how often each alert was actionable. Delete the ones that never were.

When to retrain

- **Scheduled** — weekly or monthly. Boring, predictable, and correct far more often than people expect. Start here.
- **Volume-based** — every N new labelled examples.
- **Performance-based** — metric falls below a floor. Requires labels.
- **Drift-based** — input distance exceeds a threshold. Available immediately, but noisier.

Automatic retraining needs a gate

A retrain that ships without a **validation gate** is an automated way to deploy a worse model. The new candidate must beat the incumbent on a held-out set before it is promoted — and the pipeline must be able to refuse.

Log Every Prediction You Serve

Record, per request

- A request ID and a timestamp
- The **input features as the model saw them** (post-preprocessing)
- The raw output and the final decision
- The **model version** and config version
- Latency, and any fallback that fired

Join the label in later, on the request ID. That join is what makes every metric on the previous slides computable at all.

What this buys you

- **Debugging:** replay the exact input that produced the complaint.
- **Training data:** today's production log is next quarter's training set.
- **Audit:** answer "why was this person declined, by which model, on what inputs?"

Privacy is a design input

Prediction logs contain personal data. Decide retention, access and pseudonymisation *before* you switch logging on, not after.

Why You Cannot Just Write Unit Tests

Ordinary software

- The correct output is **specified in advance**.
- `assert add(2,2) == 4`
- A test failing means the code is wrong.
- Behaviour changes only when code changes.

Machine learning

- The correct output is **learned**, and only known statistically.
- `assert model.predict(x) == ?`
- A metric dropping might mean the code is wrong, the data changed, or the seed differed.
- Behaviour changes when *data* changes, with no commit at all.

So you test three separate things

(1) the *code* around the model, with ordinary unit tests; **(2)** the *data* going in, with schema and quality checks; **(3)** the *model's behaviour*, with properties and thresholds rather than exact values.

Testing the Data

Two layers

- **Schema:** columns present, types correct, categories in the allowed set, nullability as declared. This is a hard failure — stop the pipeline.
- **Quality / statistics:** distributions within expected bounds, null rate below a ceiling, no duplicate keys, freshness within an SLA. This is usually a warning, and a gate on retraining.

Write the schema once and reuse it at three points: training input, retraining input, and serving input. That single artifact is your best defence against training/serving skew.

```
def validate(df: pd.DataFrame) -> None:
    assert set(SCHEMA) <= set(df.columns), "missing␣
           column"
    assert df["age"].between(18, 120).all()
    assert df["contract"].isin(CONTRACTS).all()
    assert df["monthly_charges"].isna().mean() < 0.01
    assert df["id"].is_unique
    assert (NOW - df["event_ts"].max()).days < 2   #
           freshness
```

Tools that formalise this: Great Expectations, Pandera, TensorFlow Data Validation, `dbt` tests. Twenty lines of `assert` statements already capture most of the value.

Testing the Model

Test type	What it asserts
Smoke / shape	The model loads, accepts one row, returns a probability in $[0, 1]$. Catches most deployment breakage.
Minimum functionality	Cases so easy that failure is unambiguous: a customer who cancelled last month must score high risk.
Invariance	Perturbations that must *not* change the prediction: reordering columns, changing a customer ID, whitespace in a category.
Directional	Perturbations with a known sign: raising the price should not *lower* predicted churn.
Golden set	A fixed, curated set of examples with expected outputs, reviewed by a human and version-controlled.
Regression / slice	Overall metric above a floor *and* no important segment worse than the incumbent by more than δ.

- **Slice tests matter more than the headline metric.** An overall $+1\%$ that hides -8% for one region is a regression, not an improvement.
- The invariance/directional/minimum-functionality vocabulary is from Ribeiro et al., "Beyond Accuracy: Behavioral Testing of NLP Models with CheckList," ACL 2020, arXiv:2005.04118v1 — the idea transfers directly to tabular models.

A Rubric to Audit Yourself: The ML Test Score

Breck et al., IEEE Big Data 2017

28 actionable tests for production readiness, seven in each of four sections: **Features and Data**, **Model Development**, **ML Infrastructure**, and **Monitoring**.

How it is scored

- **Half a point** per test executed manually, with the result documented and distributed.
- **A full point** if a system runs that test automatically and repeatedly.
- Sum within each of the four sections.
- **The final score is the minimum of the four section scores.**

Why the minimum?

- Because all four areas are necessary. Beautiful model development with zero monitoring scores zero.
- It is deliberately hard to game by doing more of what you already enjoy.

Run this on your own project this week. Most teams score well under 1 — and the exercise tells you exactly which section to invest in next.

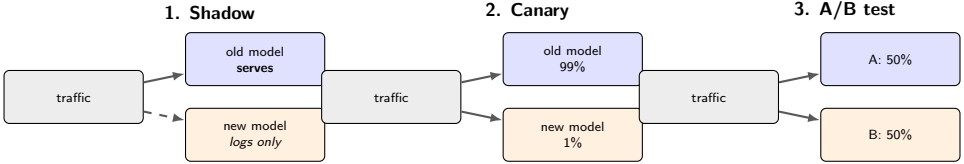
Source: Breck, Cai, Nielsen, Salib & Sculley, "The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction," *Proc. IEEE Big Data*, 2017. Paper.

CI/CD for ML: What Runs When

Trigger	What runs	Gate
Every commit / PR	Linting, unit tests, a training run on a *tiny* sample	Must be green to merge; must finish in minutes
Merge to main	Build the container, publish it by digest	Image builds and the smoke test passes
New data or a schedule	Full training pipeline, with data validation first	Candidate beats the incumbent on the frozen evaluation set
Manual promotion	Register the model version, then staged rollout	A human approves — or an automated policy does, with a rollback plan

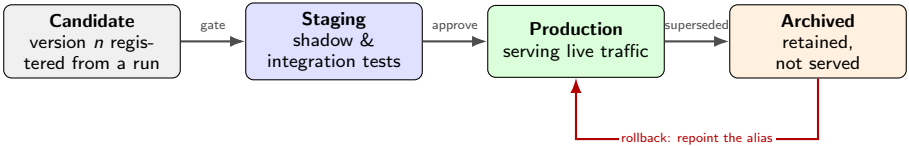
- **Keep the per-commit job fast.** A CI job that trains the real model for six hours is a CI job people learn to skip.
- **Continuous training** (retraining on fresh data) and **continuous delivery** (shipping new pipeline code) are different loops running at different cadences. Do not entangle them.

Never Flip the Switch: Shadow, Canary, A/B



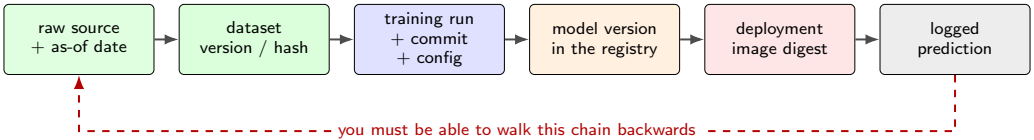
- **Shadow:** the new model sees real traffic and its predictions are logged and compared, but never returned. Catches crashes, latency blow-ups and wild disagreement at **zero user risk**. Do this first, always.
- **Canary:** a small slice of real traffic gets the new model. Watch error rate, latency and the business metric; ramp 1% → 10% → 50%, with an automatic rollback rule written down *before* you start.
- **A/B test:** a designed experiment to measure whether the new model actually improves the *business* outcome — which is not the same question as whether its F1 is higher.
- **Rollback must be one command and it must be rehearsed.** The previous image digest and the previous model version are both still in the registry: that is what they are for.

The Model Registry: A Lifecycle, Not a Folder



- A registry stores **named models** with **numbered versions**, each carrying tags, metrics and a pointer back to the run that produced it.
- Deployments should reference a **mutable alias** (“`champion`”, “`challenger`”) that points at an **immutable version**. Promotion is then a metadata change, not a redeploy — and rollback is repointing the alias.
- MLflow implements exactly this with registered models, versions, aliases and tags. (Older material describes fixed “stages”; aliases and tags are the current mechanism.)
- **Never delete the previous production version.** It is your rollback target and your audit record.

Lineage: Answering "Where Did This Number Come From?"



- The chain is only as strong as its weakest link. One un-versioned step — "we pulled the customers table" — breaks the whole trace.
- **Two questions a customer, a reviewer or your own on-call will ask:** which model produced *this* decision, and what data was it trained on? Both must be answerable in minutes, from a log, without a person's memory.
- Nothing exotic is required: an ID at each hop, stored with the next hop. Experiment tracking plus a registry plus prediction logs gives you the whole chain.

An Incident, Start to Finish

What happened

1. Tuesday 09:14 — the manual-review queue triples. No alert fires; the service is healthy.
2. 11:40 — an analyst notices. Predicted-positive rate went from 4% to 19% overnight.
3. 12:20 — the prediction log shows `tenure_months` is 0 for 80% of requests since 02:00.
4. 12:35 — an upstream job began writing nulls; the serving code imputed them to zero, silently.

What made it survivable

- **Prediction logs** contained the post-preprocessing features — so the cause was found in minutes, not weeks.
- **Model version** was on every record, ruling the model out immediately.
- **The previous version was still in the registry**, so a rollback was one alias change.

What was missing: a null-rate check on serving inputs, and an alert on prediction distribution. Both were one-day fixes.

The lesson

The model was never wrong. **Rolling back the model would not have fixed anything** — which is exactly why you need the lineage to tell you where to look.

Documentation and the Hand-off to Governance

Ship documentation as an artifact

- A **model card** — intended use, training data, disaggregated performance, known limitations, owner — registered alongside the model version, not written in a wiki that rots.
- A **datasheet** — the same discipline for the dataset it was trained on.
- Generate what you can **automatically** from the tracking run: metrics, data version, config, commit. Only the judgement calls need a human.

The content and ethics of these documents were covered in depth earlier in the course; here we only care that the pipeline *produces and versions* them.

A minimal governance checklist

- Every production model has a named **owner** and a documented purpose.
- Every model has a **review date**; unreviewed models get retired.
- Decisions affecting people are **logged and explainable**.
- There is a written **escalation path** for a contested prediction.
- Retraining and promotion require a recorded **approval**.

Mitchell et al., "Model Cards for Model Reporting," FAT* 2019, arXiv:1810.03993v2; Gebru et al., "Datasheets for Datasets," *Communications of the ACM* 64(12), 2021, arXiv:1803.09010v8.

Hand-off: bias auditing, fairness definitions and interpretability belong to the responsible-AI material seen earlier. MLOps supplies the *mechanisms* — lineage, prediction logs, versioned artifacts — that make those commitments enforceable rather than aspirational.

Summary of the Lecture

- **The ML code is the small box.** Most of the cost and most of the risk in a deployed model live in the infrastructure around it (Sculley et al., 2015).
- **The lifecycle is a loop**, not a line: experiment → validate → package → deploy → monitor → retrain. MLOps exists to make that loop cheap enough to run often.
- **Reproducibility** comes from seeds, pinned environments, versioned data and declared configuration — and it is what makes rollback possible.
- **Experiment tracking** turns a run into a first-class, append-only object with parameters, metrics and artifacts. MLflow: Tracking, Projects, Models, Model Registry.
- **Hyperparameter optimisation** beyond grid and random means *using* past trials (TPE) and *stopping* bad ones (successive halving / ASHA). Pruning is usually the bigger win.
- **Serving** is a choice between batch, online and streaming — default to batch. Serialise the whole pipeline, and never load an untrusted pickle.

Summary of the Lecture (cont.)

- **Monitoring** distinguishes upstream breakage, data drift and concept drift. Labels lag, so watch inputs and prediction distributions in the meantime.
- **Testing, staged rollout and lineage** are what let you ship a change without betting the product on it.

If You Do Only Eight Things

A weekend of work that removes most of the pain, for any project, at any scale.

This week

1. Set every seed, and log it.
2. Commit a **lockfile**; log its hash with each run.
3. Write data to **immutable dated paths**; never overwrite.
4. Start a local **MLflow** server and log every run, including the failures.

This month

5. Add a **schema check** at training *and* serving input.
6. Serialise the **whole pipeline** as one artifact and register it by version.
7. **Log every prediction** with its model version.
8. Add **one** alert: prediction distribution versus a fixed reference window.

The point

None of this requires a platform team, a cloud budget, or a new job title. It requires deciding that the run, the data and the model are **artifacts with identities** — and everything else follows.

Where to Practise This

Exercise 1 — Tracking and tuning

Wrap an existing training notebook in **MLflow** tracking; drive it with an **Optuna** study using a pruner; compare the runs in the UI; register the best model.

Entirely local — no accounts, no cloud, no cost.

Exercise 2 — Serving and drift

Export the trained pipeline, serve it with **FastAPI**, then simulate drift by shifting and corrupting the inputs; build a simple drift detector and alert on the shift in the prediction distribution.

- **Then apply it to your own project.** Score yourself with the **ML Test Score** rubric and fix whichever of the four sections is lowest.
- **Where this continues:** governance, model cards and fairness auditing were covered earlier in this course. The training-loop reproducibility mechanics are in the computer-vision course; inference optimisation and distributed serving, for models too large for a single CPU, in the NLP course.

References

1. Sculley, Holt, Golovin, Davydov, Phillips, Ebner, Chaudhary, Young, Crespo & Dennison, "Hidden Technical Debt in Machine Learning Systems," *Advances in Neural Information Processing Systems 28* (NIPS 2015). Paper (Figure 1 — the "small box").
2. Sculley et al., "Machine Learning: The High-Interest Credit Card of Technical Debt," SE4ML Workshop, NIPS 2014. Paper (the earlier, longer version of the same argument).
3. Breck, Cai, Nielsen, Salib & Sculley, "The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction," *Proc. IEEE Big Data*, 2017. Paper.
4. Google Cloud, "MLOps: Continuous delivery and automation pipelines in machine learning," Cloud Architecture Center, 2020. Article (maturity levels 0/1/2; source of the "elements of ML" figure).
5. Amershi, Begel, Bird, DeLine, Gall, Kamar, Nagappan, Nushi & Zimmermann, "Software Engineering for Machine Learning: A Case Study," ICSE-SEIP 2019. Paper (the nine-stage workflow).
6. Paleyes, Urma & Lawrence, "Challenges in Deploying Machine Learning: a Survey of Case Studies," *ACM Computing Surveys*, 2022. arXiv:2011.09926v3.
7. Zinkevich, "Rules of Machine Learning: Best Practices for ML Engineering," Google. Guide (43 rules; the best single practitioner document on this topic).
8. Zaharia, Chen, Davidson, Ghodsi, Hong, Konwinski, Murching, Nykodym, Ogilvie, Parkhe, Xie & Zumar, "Accelerating the Machine Learning Lifecycle with MLflow," *IEEE Data Engineering Bulletin* 41(4), 2018, pp. 39–45. PDF.

References (cont.)

9. MLflow documentation — “MLflow Tracking,” “MLflow Projects,” “MLflow Models” and “Model Registry.” mlflow.org/docs/latest/ml (source of the tracking-setup and tracking-UI figures).
10. Weights & Biases documentation and pricing. Docs, Pricing (accessed 2026).
11. Bergstra, Bardenet, Bengio & Kégl, “Algorithms for Hyper-Parameter Optimization,” NIPS 2011. Paper (the original TPE).
12. Bergstra & Bengio, “Random Search for Hyper-Parameter Optimization,” *JMLR* 13, 2012, pp. 281–305. Paper.
13. Akiba, Sano, Yanase, Ohta & Koyama, “Optuna: A Next-generation Hyperparameter Optimization Framework,” KDD 2019. arXiv:1907.10902v1 (source of the pruning figure, Figure 11(a)).
14. Optuna documentation — `TPESampler`, `SuccessiveHalvingPruner`, `create_study`.
15. Li, Jamieson, DeSalvo, Rostamizadeh & Talwalkar, “Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization,” *JMLR* 18, 2018. arXiv:1603.06560v4.
16. Li et al., “A System for Massively Parallel Hyperparameter Tuning,” MLSys 2020. arXiv:1810.05934v5 (ASHA).
17. Pineau, Vincent-Lamarre, Sinha, Larivière, Beygelzimer, d’Alché-Buc, Fox & Larochelle, “Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program),” *JMLR* 22(164), 2021. arXiv:2003.12206v4 (the ML Reproducibility Checklist).
18. PyTorch documentation, “Reproducibility.” docs.pytorch.org.

References (cont.)

19. scikit-learn documentation, "Model persistence." scikit-learn.org (the pickle security warning; `skops` and ONNX alternatives).
20. DVC — Data Version Control. Documentation.
21. ONNX — Open Neural Network Exchange. onnx.ai. Hugging Face, `safetensors`. Documentation.
22. Gama, Zliobaite, Bifet, Pechenizkiy & Bouchachia, "A Survey on Concept Drift Adaptation," *ACM Computing Surveys* 46(4), Article 44, 2014. DOI:10.1145/2523813 (Figure 2 — patterns of change over time).
23. Quinonero-Candela, Sugiyama, Schwaighofer & Lawrence (eds.), *Dataset Shift in Machine Learning*, MIT Press, 2009. Book.
24. Ribeiro, Wu, Guestrin & Singh, "Beyond Accuracy: Behavioral Testing of NLP Models with CheckList," ACL 2020. arXiv:2005.04118v1 (minimum functionality, invariance and directional tests).
25. Mitchell, Wu, Zaldivar, Barnes, Vasserman, Hutchinson, Spitzer, Raji & Gebru, "Model Cards for Model Reporting," FAT* 2019. arXiv:1810.03993v2.
26. Gebru, Morgenstern, Vecchione, Vaughan, Wallach, Daumé III & Crawford, "Datasheets for Datasets," *Communications of the ACM* 64(12), 2021, pp. 86–92. arXiv:1803.09010v8.
27. C. Huyen, *Designing Machine Learning Systems*, O'Reilly, 2022 — the recommended book-length treatment of everything in this deck.
28. A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed., O'Reilly, 2022, Chapter 2 and Chapter 19 (end-to-end project; training and deploying at scale).